

CompanyGraph — From one command to fully integrated

How far a company's model reaches, level by level.

RESTS ON `concept Instance`

Robert Blust · Software Engineer & Architect

One model, five levels

Model. Gated. Governed. Served. Published. Each level adds one thing that reads from the model, and none of them asks you to write it again.

RESTS ON concept Instance concept Core



Level 1 — Model: one command

5 Published

4 Served

3 Governed

2 Gated

1 Model

`init` writes the instance: core at a named release, a manifest that records it file by file, the check, the agent skills and a commit hook.

The model is a folder of Markdown. Nothing else is needed to start.

RESTS ON `feature An instance is made and kept from one command`

concept Pin

decision Core is vendored into an instance at a release its manifest names

feature An agent does the work no script can, by the skills an instance ships

```
$ npx --yes 'github:companygraph/meta-model#semver:*' init acme
✓ files written into acme
  core <release>, vendored under meta/core/
  the model is empty but for its README files, its source and its
  singular entities
  run "npx github:companygraph/meta-model#<release> check acme" whenever
  it changes
```

Level 2 — Gated: every change is checked

On GitHub, the workflow runs the same check on every pull request, at the release the manifest names. Protect the main branch, and a reference that names nothing never reaches it. A repository that never reaches GitHub takes hooks in git that check every commit, ids included, before it is made.

5 Published

4 Served

3 Governed

2 Gated

1 Model

RESTS ON `feature Checks an instance runs` `concept Check`

`question Will a new CompanyGraph release break our model?`

`companygraph / companygraph ✓``conventions / conventions ✓`

Level 3 — Governed: your rules, our machinery



Every repository around the model is held to one Markdown form, declares what it takes from another in `pins.json`, and is told which of those pins are behind. A repository without a model takes the same machinery with adopt.

It is our machinery, not our rules: your seats, your repositories, your conventions.

RESTS ON

feature A repository is held to one form and told which pins are behind

concept Pin

concept Check

decision An agent's commit is authored by the seat it held

```
$ companygraph pins
Pins of acme
  current core-release companygraph/meta-model in
  .companygraph/manifest.json: <release>
```

05

Rules, risks and controls are pages too

A rule says what must, must not or may happen, and why. A risk says what could go wrong. A control says what stops it, and points at the check that does.

The control is the page; the check stays code.

RESTS ON

feature Checks an instance runs

concept Check

```
# Main requires a review
kind: preventive  mode: automated
enforces: A change is reviewed before it ships
mitigates: An unreviewed change reaches customers
```

Level 4 — Served: an agent asks the model



A read-only server answers an agent over MCP, and a chat answers a visitor, both from the model at one named commit, naming the pages each answer rests on.

RESTS ON `feature An agent reads the model`

`feature A visitor asks the model`

`decision The server only reads, and answers at one named commit`

`question Can I trust what this chat says?`

```
search "Resolution"
→ Resolution    bounded-context
    Turns the names a page writes into the edges of the graph...
```

Level 5 — Published: pages drawn from the model

The site draws its stage, its pages and their structured data from the model at a pinned commit, and the editor reads the same files. Change a page, move the pin, and the site says it.

RESTS ON `feature The vocabulary on the web`

`feature Checks while a file is edited`



A pack climbs every level

On October 2, 2026, the software pack went from a release to our own model, our first bounded context, the website, the server and the editor, one pinned step at a time.

RESTS ON

concept Pack

bounded-context Resolution



Pins hold the levels together

Each level pins the one below it: a tag for what it runs, a commit for what it shows. A pin moves when someone decides it should.

The same day, one pin we had not moved stopped every check in an organization. Moving it fixed them all.

RESTS ON `concept Pin` `concept Release` `decision A release is its tag, and nothing we make is published to a registry`

Where are you?

The levels are a ladder, not a checklist. Each one pays for itself before you take the next. Start with one command.

RESTS ON `feature An instance is made and kept from one command`

